

Assignment Brief: LLM-powered Term Sheet Reconciliation

Context

As an Applied AI Developer in our Capital Markets division, your challenge is to create an automated reconciliation workflow. The goal is to compare key data points extracted from unstructured trade term sheets (in PDF/Word formats) with structured booking system records (CSV/JSON/XML). You will leverage a Large Language Model (LLM) that offers a public, free, or developer API to automate the parsing, extraction, and comparison. The final solution must be submitted as a public GitHub repository that is reproducible and well-documented.

Assignment Tasks

1. Automated Parsing & Structured Extraction

- Connect to an LLM via a programmatic API. (free-tier or developer API)
- Automate the process so parsed document content is submitted to the LLM programmatically; extract all required fields (e.g., ISIN, coupon, maturity, issuer, currency, notional, settlement date).
- No manual copy-pasting or web interface usage- all interactions must be performed by Python code/scripts via API.
- Schema Validation: You must define **a strict schema** (e.g., using Pydantic or a JSON schema) for the extracted fields (ISIN, coupon, maturity, etc.). The extraction must be validated against this schema; any field that fails type-validation (e.g., a string where a float is expected) should be flagged as a Parsing Error rather than a Mismatch.
- Evidence: For every field extracted, the LLM must return the **exact quote/snippet** from the source document used to derive that value.

2. Automated Booking Extract Ingestion

- Parse the booking system extract (CSV/JSON) using code.

3. Intelligent Reconciliation & False Alert Analysis

- Perform an initial comparison between the structured LLM output and the booking
- For any identified "Mismatch," you must implement a second programmatic LLM pass. This pass should analyze the mismatch, the source quote from the term sheet, and the booking value to determine if it is a **True Mismatch** or a **False Alert** (e.g., a formatting difference like "10.75%" vs "0.1075").
- The final output must categorize results as: Match, False Alert (with reason), or True Mismatch.

4. Documentation

- Include a concise report (max 2 pages) explaining your workflow, LLM/API integration, field extraction design, challenges, assumptions, and suggestions for additional automation or DevOps deployment in production.

5. Repository and Submission

- Publish your complete solution as a public GitHub repository.
- The repository must include:
 - All Python source code/scripts (well-structured).
 - Sample term sheets (PDF/Word), booking system extract files
 - A clear README.md with:
 - Step-by-step instructions to set up the environment (Python version, dependencies, API keys, etc.).
 - How to run each script end-to-end for a full workflow demonstration.
 - Configuration (e.g., API key setup instructions, required environment variables).
 - Description of expected input/output files/formats.
- The required documentation file.

6. Testing & Reproducibility

- Your code should run end-to-end on the supplied mock data with just the setup steps described in your README.md.
- We will download your repository and execute the workflow for review.

Constraints

- You must use an LLM that offers a free or public developer API. All LLM/API interactions must be fully automated—no manual steps.
- All deliverables must be in your public GitHub repository, ready to clone and run.
- You have 24 hours from receiving this assignment to complete and submit the work.